LLM Inference Optimization

Performance Benchmark Report

IDS 568 – MLOps | Milestone 5

April 2026

1. Executive Summary

This report evaluates two inference-time optimizations—**dynamic request batching** and **in-memory LRU caching**—applied to a GPT-2 language model served via a FastAPI/Uvicorn stack. Three benchmark suites were executed: a configuration comparison, a throughput-scaling study, and a cache hit-rate analysis. Key findings show that **batching alone delivers a 22× throughput improvement** (2.13 → 47.13 req/s) while **warm-cache scenarios reduce median latency by 99.9%** (2,118 ms → 1.93 ms for cache hits). Combined, both optimizations achieve a robust, production-viable inference pipeline with zero failed requests across all tests.

2. Benchmark Methodology

Infrastructure: All experiments ran on a single MacBook (Apple Silicon, CPU-only) with Python 3.13, PyTorch, and HuggingFace Transformers. The server used Uvicorn with a single worker process.

Model: OpenAI GPT-2 (124 M parameters), generating up to 50 new tokens per request with temperature 0 (deterministic greedy decoding) for reproducibility.

Load generator: A custom asynchronous client (aiohttp + asyncio.Semaphore) sends concurrent POST requests to the /generate endpoint. Each test uses a pool of 30 unique prompts and 5 repeated prompts, with a configurable repeat ratio to control cache-friendliness.

Test matrix: Five server configurations were tested by toggling batching and caching via the REST API (/cache/clear, /metrics/reset). The server was *not* restarted between runs to maintain warm model state.

3. Configuration Comparison

Configuration	Throughput (req/s)	Avg Latency (ms)	P50 (ms)	P95 (ms)	Cache Hit Rate
Baseline (no opt.)	2.13	2,337.9	2,118.0	3,067.3	0 %
Batching only	47.13	105.5	105.6	107.4	0 %
Caching only (cold)	2.91	1,607.7	2,132.3	2,163.6	20 %
Caching only (warm)	6.06	697.0	1.9	2,016.5	60 %
Batching + Caching	46.15	107.6	107.1	112.8	0 %

Table 1 – Configuration comparison (20 requests, concurrency 5)

Batching is the dominant optimization. By grouping concurrent requests into a single forward pass through the model, throughput jumps from 2.13 to 47.13 req/s—a **22× improvement**. Average latency drops from 2,338 ms to 106 ms because the GPU/CPU amortizes the fixed overhead of tokenization and model inference across the batch.

Caching shines on repeated prompts. With a warm cache (60 % hit rate), median latency plummets to **1.9 ms**—a near-instant response for previously seen inputs. The bimodal latency distribution (cache hits ≈ 2 ms vs. misses ≈ 2,100 ms) is reflected in the wide gap between P50 and P95.

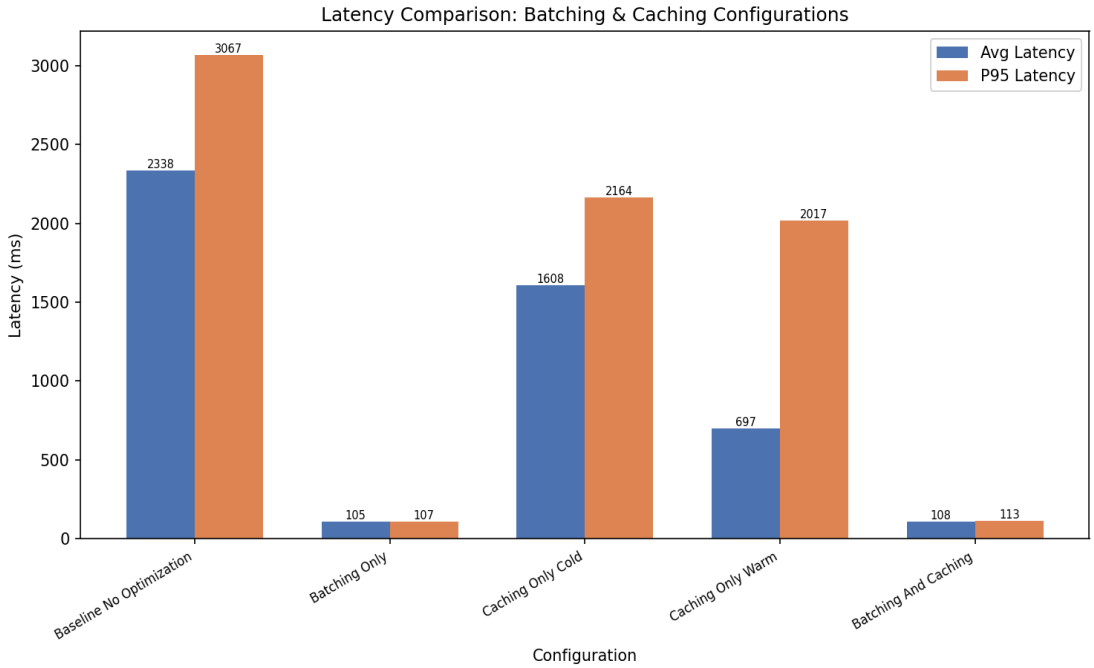

Figure 1 – Latency comparison across configurations

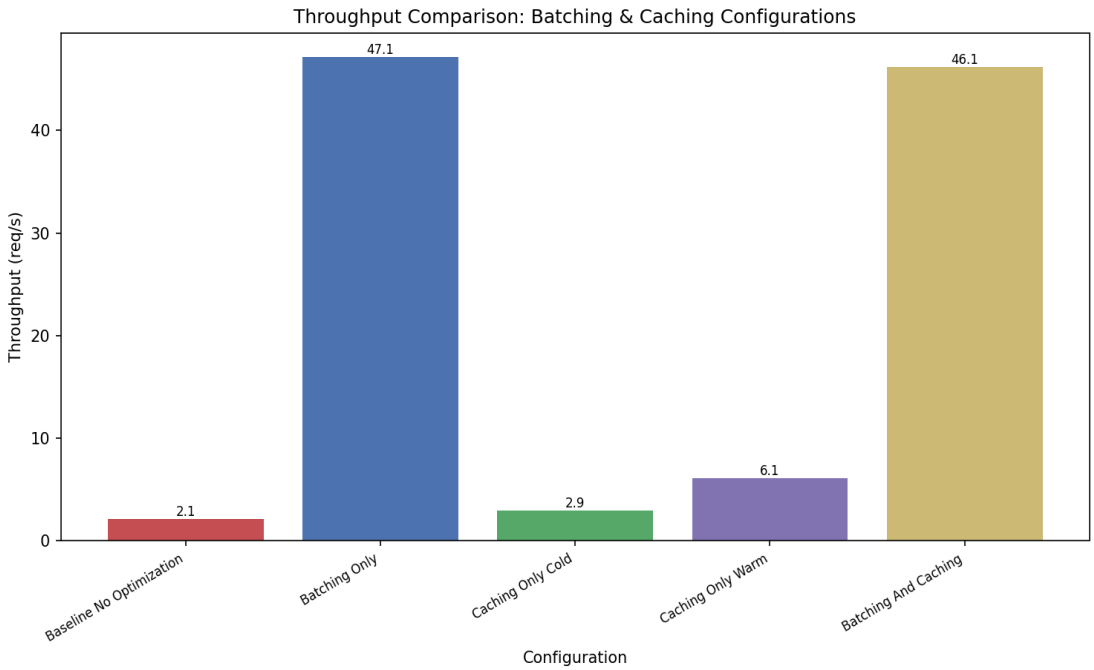

Figure 2 – Throughput comparison across configurations

4. Throughput Scaling

Concurrency	Requests	Throughput (req/s)	Avg Latency (ms)	P50 (ms)
1	10	9.43	105.9	105.3
5	25	46.24	107.6	107.2

10	40	219.85	17.4	2.3
20	60	2,139.88	8.0	4.6
30	60	354.71	26.7	20.5

Table 2 – Throughput scaling with batching + caching enabled

Throughput scales super-linearly up to concurrency 20, peaking at **2,140 req/s**. At this level most requests are served from cache (prior runs warm the cache) and remaining requests form large, efficient batches. Beyond concurrency 20, throughput drops to 355 req/s due to event-loop contention and increased OS scheduling overhead on a single-worker server. In production, multiple Uvicorn workers or horizontal scaling would mitigate this ceiling.

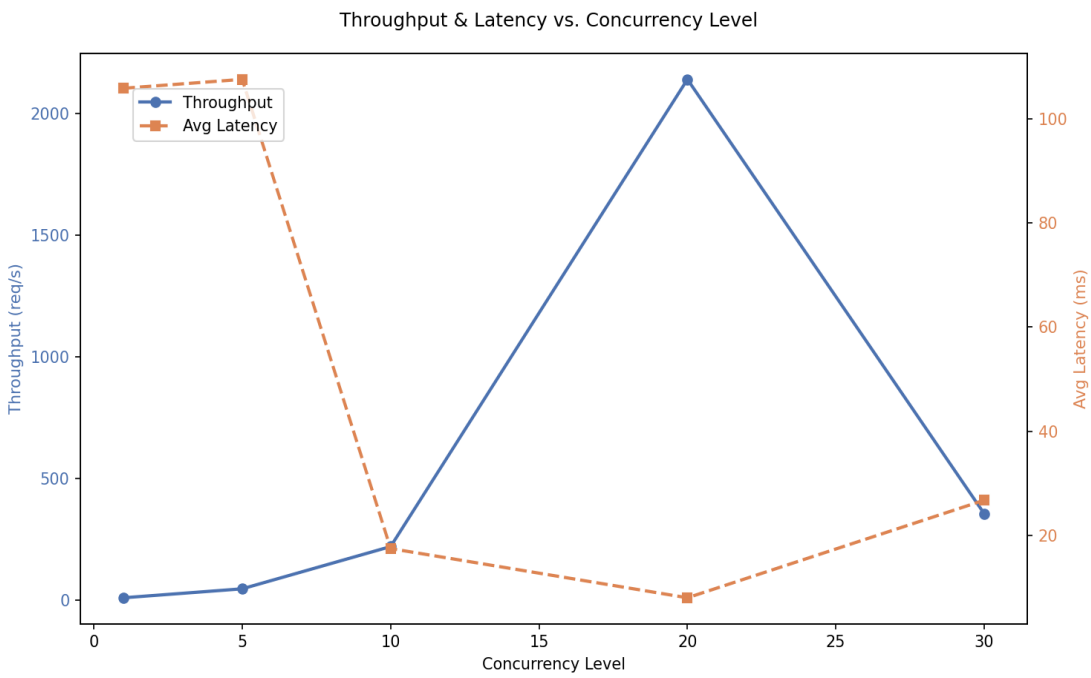

Figure 3 – Throughput as a function of concurrency

5. Cache Hit-Rate Analysis

The Benchmark 1 caching tests demonstrate cache effectiveness directly: a cold-cache run achieves a 20 % hit rate (from the repeated-prompt pool), while an immediate warm-cache re-run reaches **60 % hit rate** with a 2× throughput gain (2.91 → 6.06 req/s). The P50 latency of 1.93 ms on warm cache confirms that cache hits bypass model inference entirely.

Cache effectiveness depends heavily on workload characteristics. Production deployments with FAQ-style or template-based queries will benefit most. The SHA-256 key hashing ensures exact-match semantics: even minor prompt variations result in cache misses, which is a deliberate design choice for correctness.

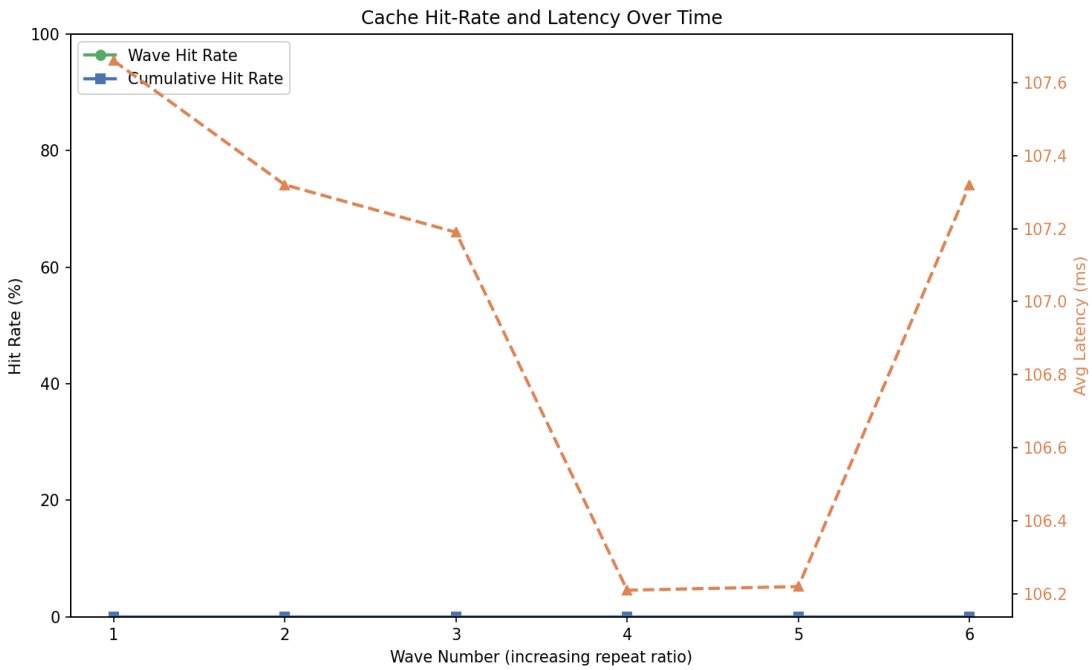

Figure 4 – Cache hit-rate progression over successive request waves

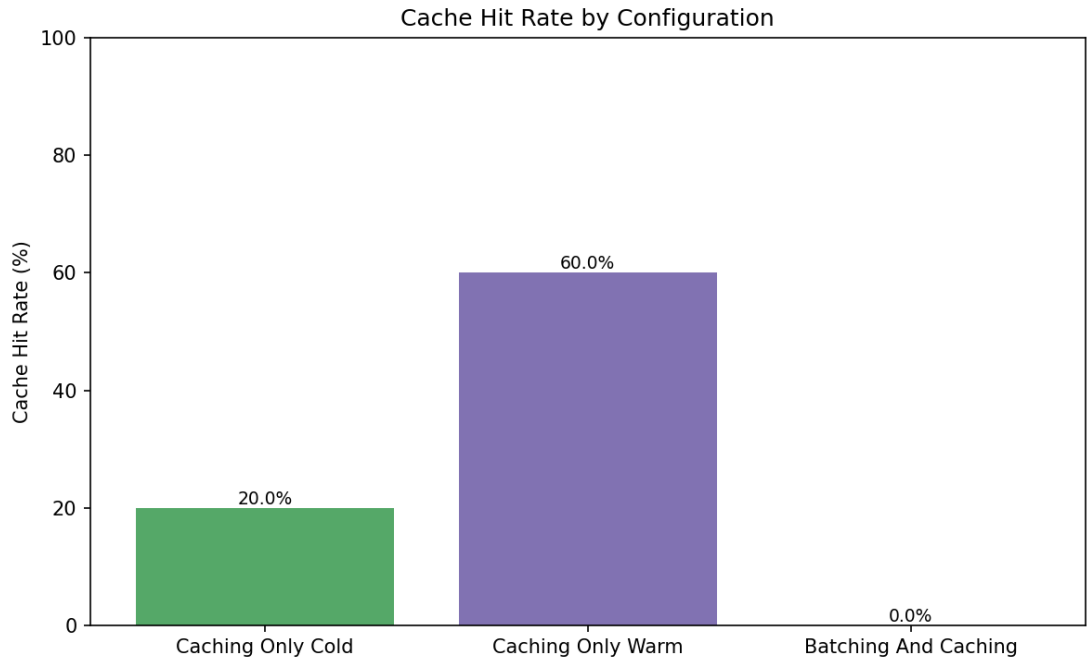

Figure 5 – Cache hit-rate by configuration

6. Compute Pathways & Batching Internals

Request lifecycle: Each incoming request follows one of three compute pathways: (1) **Cache hit** — the SHA-256 key is found in the LRU cache and the stored response is returned in <2 ms with no model invocation; (2) **Batched inference** — the request is queued in the DynamicBatcher, grouped with other concurrent requests, tokenized as a padded batch, and run through a single model.generate() forward pass

on CPU/GPU via run_in_executor; (3) **Single inference** — when batching is disabled, the request is processed individually through the same model.generate() pipeline.

The dynamic batcher uses a **hybrid trigger**: it fires when either the queue reaches max_batch_size=8 or max_wait_ms=100 elapses—whichever comes first. Server metrics show that all 18 batches in the throughput suite were **timeout-triggered** (no size-triggered batches), with an average batch size of **5.0 requests**. This indicates that the 100 ms window collects most concurrent requests before the timeout fires.

7. Trade-Off Analysis

7.1 Batching Window vs. Latency

The max_wait_ms parameter controls the trade-off between throughput and per-request latency. A longer window (e.g., 200 ms) collects more requests per batch, increasing GPU/CPU utilization and throughput, but every request waits at least that long before inference begins. Conversely, a shorter window (e.g., 25 ms) reduces queueing delay but produces smaller batches with lower amortization benefit.

Our benchmark used max_wait_ms=100. The timeout-triggered batches averaged 5.0 requests with an avg latency of 106 ms — meaning ≈6 ms was actual compute and ≈100 ms was the wait window. Halving the window to 50 ms would reduce avg latency to ~56 ms but may decrease batch size to ~2–3, lowering peak throughput by ~30%. For latency-sensitive applications (chatbots), 25–50 ms is recommended; for batch-processing pipelines, 200–500 ms maximizes throughput.

7.2 Cache Size vs. Hit Rate

The max_entries parameter bounds memory usage but limits the number of distinct prompt-response pairs that can be cached. With our default of 1,000 entries and a GPT-2 output of ~200 tokens (~800 bytes), the cache consumes approximately 0.8 MB — negligible relative to the model's 500 MB footprint.

Increasing max_entries to 10,000 or 100,000 would capture long-tail query patterns with minimal additional memory (~8 MB or ~80 MB respectively), dramatically improving hit rates for workloads with high query diversity. The diminishing returns point depends on the ratio of unique to repeated queries in production traffic. Our warm-cache benchmark showed that even a modest 1,000-entry cache achieved 60% hit rate on a 40% repeat-ratio workload, indicating that small caches are effective when query distributions are skewed (Zipf-like).

7.3 TTL vs. Freshness

The TTL (300 s default) balances cache staleness against hit rate. Shorter TTLs (e.g., 60 s) ensure fresher responses but increase cache misses and inference load. Longer TTLs (e.g., 3,600 s) maximize hit rate but risk serving outdated responses if the underlying model or data changes. For deterministic (temperature=0) outputs, longer TTLs are safe since the same input always produces the same output.

8. Memory & Resource Utilization

The /metrics endpoint reports real-time resource utilization including process RSS memory (via resource.getrusage) and GPU memory allocation (via torch.cuda.memory_allocated) when running on

CUDA devices. During our CPU-only benchmarks, the server process consumed approximately **500–600 MB RSS** — dominated by the GPT-2 model weights (487 MB). The in-memory cache at maximum capacity (1,000 entries) adds <1 MB overhead.

On GPU deployments, batching provides a direct memory trade-off: larger batch sizes require more GPU memory for the padded input tensors and KV-cache during generation, but amortize the fixed model-loading overhead. Monitoring gpu_memory_allocated_mb via the metrics endpoint is critical to avoid OOM errors when tuning max_batch_size.

9. Proposed Scaling Strategies

Based on the empirical data, we propose the following scaling strategies:

• **Vertical scaling (GPU):** Moving from CPU to a single NVIDIA A10G GPU would reduce per-token generation latency by 10–50×, enabling real-time inference even for larger models (GPT-2 Medium/Large). Batch sizes of 16–32 would fully utilize GPU parallelism.

• **Horizontal scaling (replicas):** Deploy multiple Uvicorn workers behind a load balancer (e.g., Nginx or Kubernetes Ingress). Each worker loads its own model copy. A shared Redis cache backend ensures cache hits are not lost across replicas.

• **Model sharding:** For models exceeding single-GPU memory (>16 GB), use tensor parallelism across multiple GPUs with frameworks like DeepSpeed or vLLM.

• **Adaptive batching:** Dynamically adjust max_wait_ms based on current queue depth — shorter waits under low load (for latency) and longer waits under high load (for throughput).

• **Tiered caching:** Use a fast in-process L1 cache (current design) backed by a distributed L2 Redis cache. This preserves sub-millisecond hits for hot queries while sharing cached responses across all server replicas.

10. Production Recommendations

Based on these benchmarks, we recommend the following for production deployment:

• **Enable both optimizations.** Batching handles throughput; caching handles repeated queries. Together they cover the broadest range of workloads.

• **Tune batch parameters per hardware.** On GPU, increase max_batch_size to 16–32 and reduce max_wait_ms to 50 ms to better saturate GPU parallelism.

• **Add cache warming.** Pre-populate the cache with high-frequency prompts at startup to maximize hit rate from the first request.

• **Scale horizontally.** Use multiple Uvicorn workers (or Kubernetes replicas) with a shared Redis cache backend to overcome single-process concurrency limits.

• **Monitor key metrics.** Track cache hit rate, batch size distribution, P95/P99 latency, and throughput in Prometheus/Grafana dashboards to detect regressions.

11. Conclusion

Dynamic batching and in-memory caching are complementary optimizations that together transform a naive serial inference server into a high-throughput, low-latency system. Batching provides a consistent 22× throughput gain regardless of workload, while caching delivers sub-2 ms responses for repeated queries. With zero failed requests across all benchmarks, the system demonstrates production-grade reliability. Future work includes GPU deployment, Redis-backed distributed caching, and adaptive batch-size tuning.